

Lecture 04

Integer Encodings and Data Operations

Comp_Sci 213

Introduction to Computer Systems

Monday, Oct. 5, 2026

Kate Smith – Fall 2026

Slides adapted from:

Ghena, St-Amour, Hardavellas, Bustamente (Northwestern), Bryant, O'Hallaron (CMU), Garcia, Weaver (UC Berkeley)

A note about clock frequency

The processor's clock is a steady signal that ticks at a fixed rate and coordinates when the hardware does its work

- ***Clock frequency*** (or clock rate) is the number of ticks per second, measured in hertz ($\text{Hz} = 1/\text{second}$).
 - $1 \text{ GHz} = 10^9 \text{ cycles per second}$... SI units here!
- ***Clock period*** (or cycle time) is the length of one tick
 - $\text{period} = 1 / \text{frequency}$
- Example: What is the clock period for a 2 GHz core?
 - $1/2 \text{ GHz} = 1 / (2 * 10^9 \text{ 1/s}) = 0.5 * 10^{-9} \text{ s} = 0.5 \text{ ns}$
- A higher frequency means more cycles per second, but it doesn't by itself mean a faster program!
 - Performance also depends on how much work gets done each cycle
 - $\text{Execution time} = (\text{number of operations}) / (\text{operations per cycle} * \text{cycles per second})$

Topics

- Integer Encodings
- Integer Operations

Topics

- **Integer Encodings**
- Integer Operations

Integer types in C

- C types describe both encoding and size
- Encoding
 - **signed** or **unsigned**
 - Defaults to signed if no label included
- Sizes
 - 1 byte (8-bit): **char**
 - 2 bytes (16-bit): **short**
 - 4 bytes (32-bit): **int**
 - 8 bytes (64-bit): **long**

Examples:

- **unsigned int**
 - 32-bit unsigned integer
- **signed short**
 - 16-bit signed integer
- **char**
 - 8-bit signed integer

Sizes of C types are technically system dependent

- C only defines types as a “minimum” size
 - They could be different on different computers
 - This is not a good choice
 - You should assume the x86-64 rules
 - Memorize these! Matches prior slide
- How I program
 - Use fixed width integer types `<stdint.h>`
 - `int8_t`, `int16_t`, `int32_t`
 - `uint8_t`, `uint16_t`, `uint32_t`

C Data Type	x86-64 (Bytes)	C Standard* (C99)
char	1	≥ 1
short	2	≥ 2
int	4	≥ 2
long	8	≥ 4
long long	8	≥ 8
float	4	
double	8	
pointer	8	Widths for data, code pointers may differ!

CS213 uses this



Review: pointers

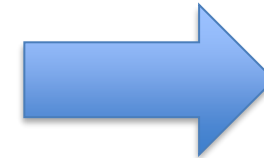
A ***pointer*** is an address and a type telling C how to use that address

```
#include <stdio.h>
int main(void) {

    int x = 42; // Get the address of x
    int *p = &x;

    printf("x = %d\n", x);
    printf("Address of x = %p\n", (void *)&x);
    printf("p = %p\n", (void *)p);
    printf("Value pointed to by p = %d\n", *p);

    return 0;
}
```



Output:

x = 42

Address of x = 0x16f3e...

p = 0x16f3e...

Value pointed to by p = 42

***Note: The (void *) cast is the conventional way to pass a pointer to printf for %p*

Expressing C types in bits

- Two families of encodings to express integers using bits
 - ***Unsigned*** encoding for unsigned integers
 - ***Two's complement*** encoding for signed integers
- Each encoding will use a fixed size (# of bits)
 - For a given machine
 - Size + encoding family determine which C type we're representing
 - Reverse is also true: a C type specifies both size and encoding
 - Fixed size is because computers are finite!

Unsigned integer encoding

- Just write out the number in binary
 - Works for 0 and all positive integers
- Example: encode 104_{10} as an **unsigned** 8-bit integer
 - $104_{10} = 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$

$\Rightarrow$ **0b01101000**

$\Rightarrow$ **0x68**

Formal Definition

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

(Binary To Unsigned)

Bounds of unsigned integers

- For a fixed width **w**, a limited range of integers can be expressed
 - Smallest value (we will call **UMin**):
 - all 0s bit pattern: 000...0, value of 0
 - Largest value (we will call **UMax**):
 - all 1s bit pattern: 111...1, value of $2^w - 1$
 - $2^w - 1 = 1 \times 2^{w-1} + 1 \times 2^{w-2} + \dots + 1 \times 2^1 + 1 \times 2^0 = 11111\dots$
- Maximum 8-bit number = $2^8 - 1 = 256 - 1 = 255$

Encoding signed integers

- What's different about representing a signed number?
 - It can be negative!
- So, we're going to have to somehow represent values that are negative and positive
- There are actually many different encodings capable of doing this
 - This is when that “nice encoding” versus “annoying encoding” matters
 - We'll focus on the most-commonly used method: two's complement

Doesn't work: attempting signed encoding

- Goal: encode integers that can be positive or negative
- First attempt: we can use the most significant bit for sign
 - “Sign-and-magnitude” encoding
 - In 8-bits:

• +4 = 00000100	+127 = 01111111	+0 = 00000000
• -4 = 10000100	-127 = 11111111	-0 = 10000000
- Annoying problem: we have two representations of zero!
- Also annoying: hardware to do math with signed and unsigned numbers gets complicated...

Two's complement encoding

- Bad news: need to make the encoding more complicated to handle negatives
- Plan:
 - Start with unsigned encoding, but then make ONLY the largest power negative
 - Example: for 8 bits, most significant bit is worth -2^7 not $+2^7$ (other bits are still positive)
- To encode a negative integer
 - First, set the most significant bit to 1 to start with a big negative number
 - Then, add positive powers of 2 (the other bits) to “get back” to number we want
- Example: encode -6 as a 4-bit two's complement integer
 - $-6_{10} = 1 \times -2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \Rightarrow 0b1010 \Rightarrow 0\mathbf{xa}$

Two's complement examples

- Encode -100 as an 8-bit two's complement number

- $-100_{10} = 1 \times -2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$

$$\begin{array}{cccccccc} -128 & +0 & +0 & +16 & +8 & +4 & +0 & +0 \end{array}$$

Problem becomes:

encode +28 as a 7-bit unsigned number

- $-100_{10} = 0b10011100 = 0x9C$
- **Shortcut:** determine positive version of number, flip all the bits, and add one
 - $100_{10} = 0b01100100$
 - Flipped = $0b10011011$
 - Plus 1 = $0b10011100 = 0x9C$ We'll talk about binary addition next lecture

Interpreting binary signed values

Formal Definition

- Converting binary to signed:

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

Sign bit

- Note: most significant bit tells us sign!! 1-> negative
 - Checking if a number is negative is just checking that top bit
- Only one encoding exists for zero – that's helpful!
 - 0b00000000 = 0 0b10000000 = -128
- -1: 0b111...1 = -1 (regardless of number of bits!)

Bounds of two's complement integers

- For a fixed width **w**, a limited range of integers can be expressed
 - Smallest value, most negative (we will call **TMin**):
 - 1 followed by all 0s bit pattern: $100\dots0 = -2^{w-1}$
 - Largest value, most positive (we will call **TMax**):
 - 0 followed by all 1s bit pattern: $01\dots1$, value of $2^{w-1} - 1$
- Beware the asymmetry! Bigger negative number than positive

Ranges for different bit amounts

	W			
	8	16	32	64
UMax	255	65,535	4,294,967,295	18,446,744,073,709,551,615
Umin	0	0	0	0
TMax	127	32,767	2,147,483,647	9,223,372,036,854,775,807
TMin	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808

- Observations

- $|TMin| = TMax + 1$
 - Asymmetric range
- $Umax = 2 * TMax + 1$

- C Programming

- `#include <limits.h>`
- Declares constants, e.g.,
 - `ULONG_MAX`
 - `LONG_MAX`
 - `LONG_MIN`
- Values are platform specific

Unsigned & Signed Numeric Values

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

- **Equivalence**
 - Same encodings for non-negative values
- **Uniqueness**
 - Every bit pattern represents a single, unique integer value
 - Each representable integer has a single, unique bit encoding
- **⇒ Can Invert Mappings**
 - Can go from bits to number and back, and vice versa

Practice

What range of integers can be represented with 5-bit two's complement?

- A. -31 to +31
- B. -15 to +15
- C. 0 to +31
- D. -16 to +15
- E. -32 to +31

Practice

What range of integers can be represented with 5-bit two's complement?

- A. -31 to +31 No asymmetry & 6-bits
- B. -15 to +15 No asymmetry
- C. 0 to +31 Unsigned
- D. -16 to +15 Correct
- E. -32 to +31 6-bits

Casting signed to unsigned

- C allows conversions from signed to unsigned (and vice versa)

```
short int      x = 15213;
unsigned short int ux = (unsigned short) x;
short int      y = -15213;
unsigned short int uy = y; /* implicit cast! */
```

- Resulting value
 - Not based on current decimal value: instead keep the bits and **reinterpret** them!
 - Non-negative values unchanged
 - $ux = 15213$
 - Negative values change into (large) positive values (and vice versa)
 - $uy = 50323$
- Warning: Casts can be implicit in assignments or function calls!

Mapping Signed $\leftrightarrow$ Unsigned (4 bits)

Large negative
value becomes
large positive!

Bits	Signed		Unsigned
0000	0	$\longleftrightarrow$ =	0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5		5
0110	6		6
0111	7		7
1000	-8	$\xrightarrow{+16 \text{ (i.e., } 2^4)}$ $\xleftarrow{-16 \text{ (i.e., } 2^4)}$	8
1001	-7		9
1010	-6		10
1011	-5		11
1100	-4		12
1101	-3		13
1110	-2		14
1111	-1		15

Example

- Convert signed 8-bit number -120 into an unsigned 8-bit number

1. Convert -120 into binary

2. Convert binary back into unsigned decimal

Example

- Convert signed 8-bit number -120 into an unsigned number
 1. Convert -120 into binary
 $-120 = -128 + 8 =$
 2. Convert binary back into unsigned decimal

Example

- Convert signed 8-bit number -120 into an unsigned number

1. Convert -120 into binary

$$-120 = -128 + 8 =$$

$$1 \times (-128) + 0 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 0 \times 1$$

0b 1000 1000

2. Convert binary back into unsigned decimal

Example

- Convert signed 8-bit number -120 into an unsigned number

1. Convert -120 into binary

$$-120 = -128 + 8 =$$

$$1 \times (-128) + 0 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 0 \times 1$$

$$0b\ 1000\ 1000$$

2. Convert binary back into unsigned decimal

$$1 \times 128 + 0 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 0 \times 1$$

$$128 + 8 = 136$$

Why convert bit length?

- Hardware has physical limits
 - We cannot hold an infinite number of bits
- So sometimes we have to change the sizes of values
 - Reducing to use fewer bits: Truncation
 - Expanding to use more bits: Extension
 - Computer hardware automatically follows these rules
 - So programming languages automatically do them as well
- These aren't mathematical operations, they're hardware limitations
 - So they might not work as you expected

Truncation (reducing number of bits)

- Truncation rule:
 - Just delete most-significant bits until the value is the correct size
 - Same rule no matter the encoding
- If the value fits in both bit sizes, it remains the same
- If not, it will be totally different
 - Result becomes identical to modular arithmetic
 - But we can always just do the truncation and check result
- 6-bit to 2-bit truncation
 - $0b000010 \rightarrow 0b\text{~~0000~~}10 = 0b10$ (decimal 2 becomes decimal 2)
 - $0b110010 \rightarrow 0b\text{~~1100~~}10 = 0b10$ (decimal 50 becomes decimal 2)
Identical to 50 modulo 4 = 2

Aside: Why do truncation?

- It can totally mess up the value!! So why do it?
- Because you sometimes have to
 - You might have a 32-bit result, but only be able to store an 8-bit value
 - Good code checks whether the result is in-bounds, and takes some action if it isn't
 - Maybe just set to maximum
 - Maybe throw an error

Extension (increasing number of bits)

- Extension rule:
 - Just add most-significant bits until the value is the correct size
- But should we add zeros or ones??
 - Unsigned extension: add zeros
 - Signed extension: add copies of most-significant bit (0 or 1)
 - Both results preserve the original value!
- 4-bit to 7-bit extension
 - Unsigned 0b0110 -> 0b0000110 (decimal 6 stays decimal 6)
 - Unsigned 0b1000 -> 0b0001000 (decimal 8 stays decimal 8)
 - Signed 0b0110 -> 0b0000110 (decimal 6 stays decimal 6)
 - Signed 0b1000 -> 0b1111000 (decimal -8 stays decimal -8 = -64+32+16+8)

Practice

- Convert 16-bit 0x3427 to an 8-bit signed integer (hex)
- Convert 8-bit 0xF0 to a 16-bit signed integer (hex)

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice

- Convert 16-bit 0x3427 to an 8-bit signed integer (hex)
 - Process: truncate extra bits
 - Answer is **0x27**
- Convert 8-bit 0xF0 to a 16-bit signed integer (hex)

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice

- Convert 16-bit 0x3427 to an 8-bit signed integer (hex)
 - Process: truncate extra bits
 - Answer is **0x27**
- Convert 8-bit 0xF0 to a 16-bit signed integer (hex)
 - Process: sign extend. Is the most-significant bit one? Yes!
 - Answer is **0xFFFF0**

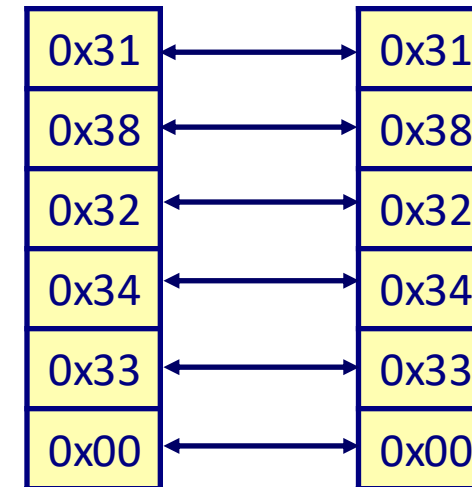
Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Encoding strings (The C way)

- Represented by array of characters
 - Each character encoded in ASCII format
 - NULL character (code 0) to mark the end
- Compatibility
 - Byte ordering not an issue (data all single-byte!)
 - ASCII text files generally platform independent
 - Except for different conventions of line termination character(s)!

```
char S[6] = "18243";
```

Big-Endian Little-Endian



Encoding color

- RGB colors
 - 3-byte values
 - First byte is Red, then Green, then Blue
- Usually specified in hexadecimal
 - #FF0000 -> maximum red, zero green or blue
 - #4E2A84 -> 1/4 red, 1/8 blue, 1/2 green (Northwestern Purple)
- 2^{24} possible colors = 16777216 colors

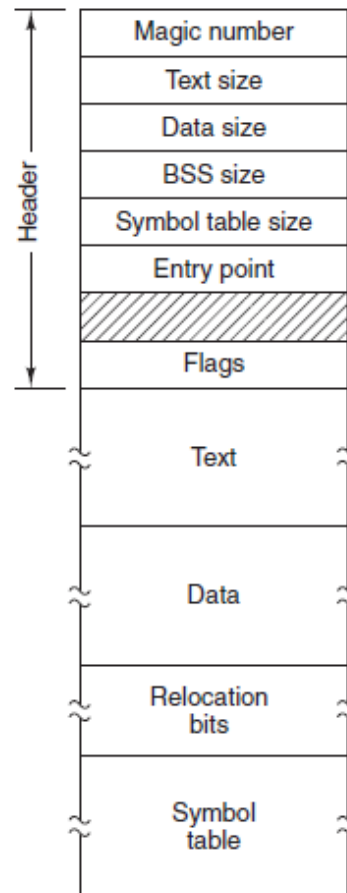


Interpreting file contents

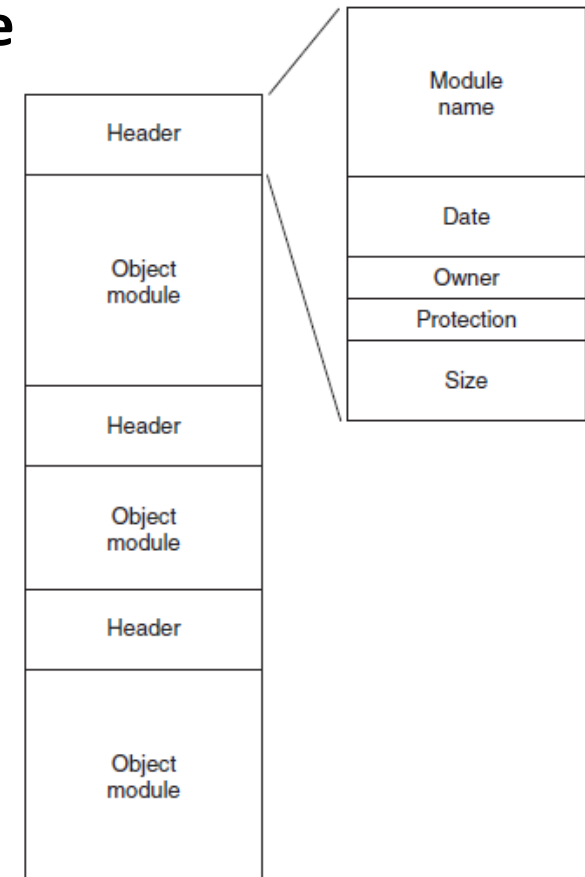
- Collections of data
 - Usually in permanent storage on your computer
- Regular files
 - Arbitrary data
 - Think of as a big array of bytes
- Non-regular files would be directories, symbolic links, or other less used things

What about different types of regular files?

- Text files versus Executables versus Tar files
 - All just differing patterns of bytes!
 - It really is just all data. The meaning is in how you interpret it.



Executable File



Archive (tar)

Identifying regular files

- `file` command can help you determine a file's type

```
Kates-MacBook-Pro:to-read katesmith$ file walking-surface-code.pdf
walking-surface-code.pdf: PDF document, version 1.5
[Kates-MacBook-Pro:to-read katesmith$ file Thesis_Defense_2023_12_04_v05.pptx
Thesis_Defense_2023_12_04_v05.pptx: Microsoft PowerPoint 2007+
Kates-MacBook-Pro:to-read katesmith$ █
```

Encoding time

- Unix time:
 - 32-bit signed integer counting seconds elapsed since initial time
 - Initial time was January 1st at midnight UTC, 1970
- Current Unix time (when this slide was last edited):
 - Negative numbers would mean times before 1970
 - <https://www.unixtimestamp.com/>
- Problem: when does Unix time hit the maximum value?
 - 2147483647 seconds from January 1st 1970
 - Result: January 19th, 2038
 - This is the “[Year 2038 Problem](#)”

1791207805
SECONDS SINCE JAN 01 1970. (UTC)
8:43:27 AM

Topics

- Integer Encodings
- **Integer Operations**

C versus the hardware

- Operations you can perform on types have edge conditions
 - Usually going above or below the bounds for the given size/encoding
- If we say what happens in that scenario, it'll be what “the hardware” (i.e., a computer) does
 - In today's examples, pretty much every computer does the same thing
- That is not the same as what C does
 - Unclear choices are left as **undefined behavior**
 - **Undefined Behavior:** C standard places no requirement on what happens if a program does that thing
 - The compiler can make any choice it wants
 - Program might crash, produce garbage, appear to work, behave differently based on optimization level, etc.

Unsigned Addition

- Like grade-school addition, but in base 2, and ignores final carry
 - If you want, can do addition in base 10 and convert to base 2. Same result! But here we're going to understand what the hardware is doing.
- **Example: Adding two 4-bit numbers**

$$\begin{array}{r} \overset{1}{0}\overset{1}{1}\overset{1}{0}1 \\ + \quad 0011 \\ \hline 1000 \end{array}$$

- $5_{10} + 3_{10} = 8_{10}$ ✓

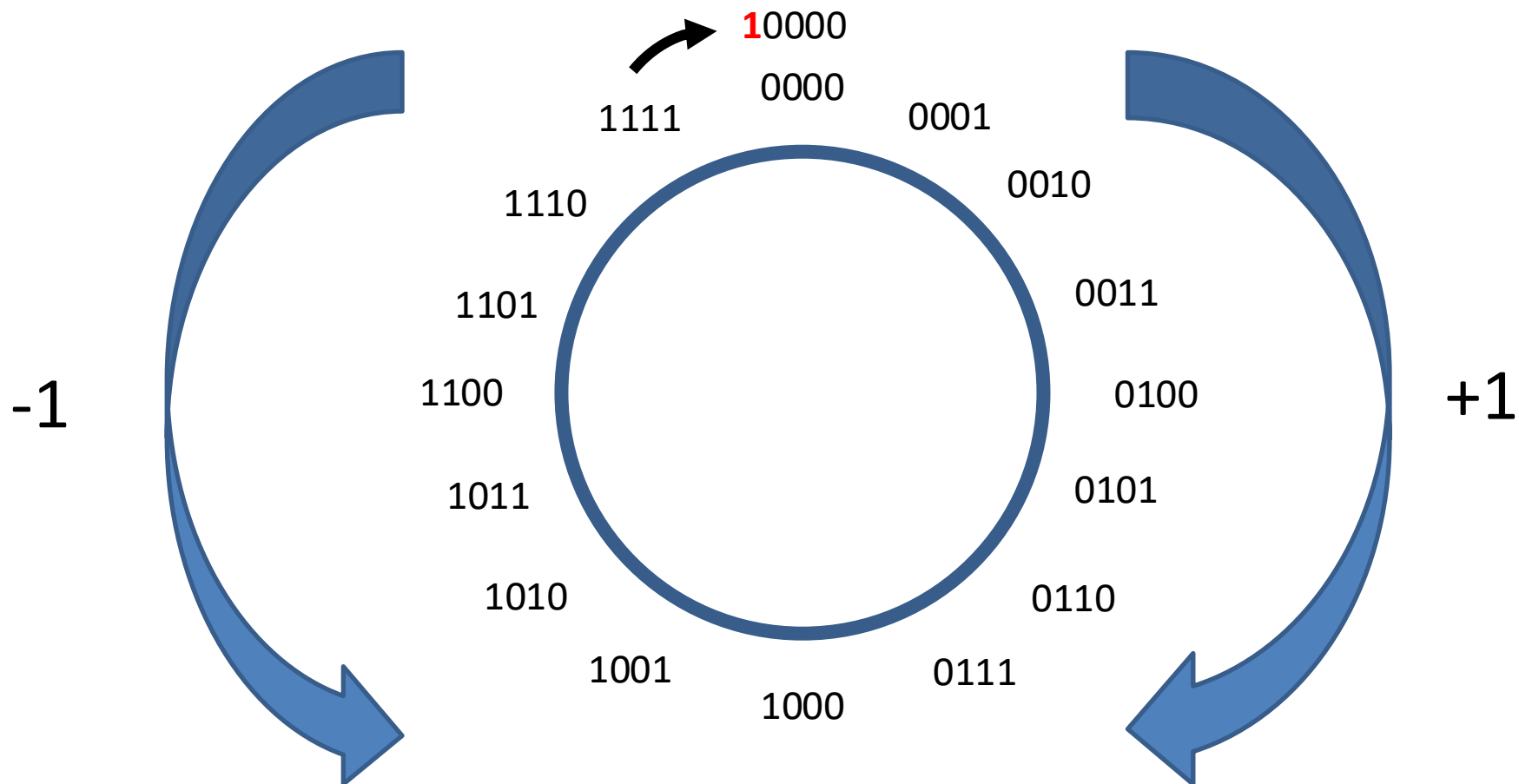
Unsigned Addition and Overflow

- What happens if the numbers get too big?
- **Example: Adding two 4-bit numbers**

$$\begin{array}{r} \overset{1}{1} \overset{1}{1} \overset{1}{1} \overset{1}{1} \\ 1101 \\ + 0011 \\ \hline 10000 \end{array}$$

- $13_{10} + 3_{10} = 16_{10}$
 - Too large for 4 bits!
 - Truncate most-significant bits that do not fit
 - Gives us modular (= modulo) behavior: $16 \text{ modulo } 2^4 = 0$
 - Final result is the 4 least significant bits (all we can fit): so 0_{10}

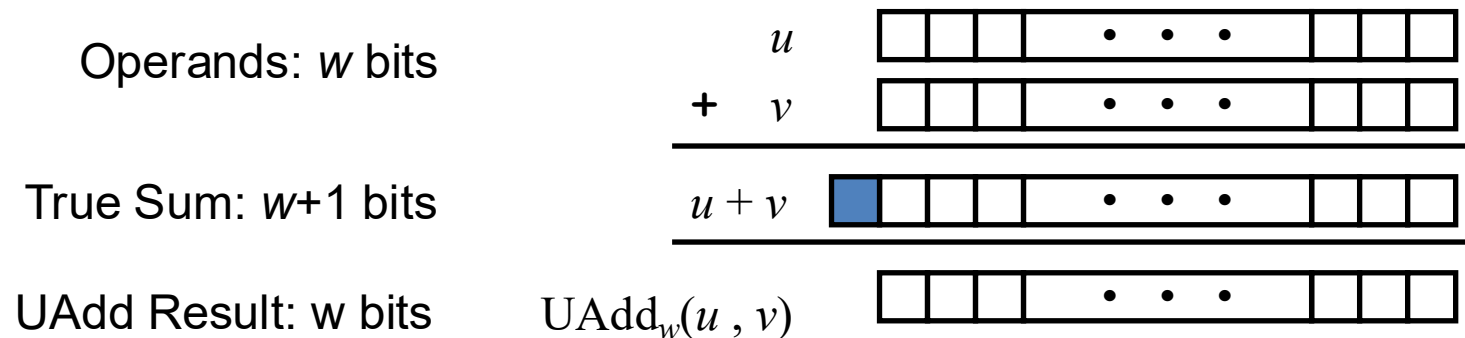
Modulo behavior in binary numbers



****Note:**
 $N \bmod r$ ($N \% r$) will be
some value $[0, r-1]$

Unsigned addition is modular

- Implements modular arithmetic
 - $\text{UAdd}_w(u, v) = (u + v) \bmod 2^w$
- Need to drop carry-out bit, otherwise results will keep getting bigger
 - Example in base 10: $80_{10} + 40_{10} = 120_{10}$ (2-digit inputs become a 3-digit output!)



- Warning: C does not tell you that the result was truncated!
 - **Unsigned** addition in C silently truncates most-significant bits beyond the limit

Signed (2's Complement) Addition

- Works exactly the same as unsigned addition!
 - Just add the numbers in binary, and the result will work out
- Signed and unsigned sum have the exact same bit-level representation
 - Computers use the same machine instruction and the same hardware!
 - That's a big reason 2's complement is so nice! Shares operations with unsigned

Signed addition example

- Same addition method as unsigned
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{rcl} & \overset{1}{1} & \overset{1}{1} \\ & 10\bar{1}1 & (-5) \\ + & 0011 & (+3) \\ \hline & 1110 & (-2) \end{array}$$

○ $-5_{10} + 3_{10} = -2_{10} \checkmark$

Combining negative and positive numbers

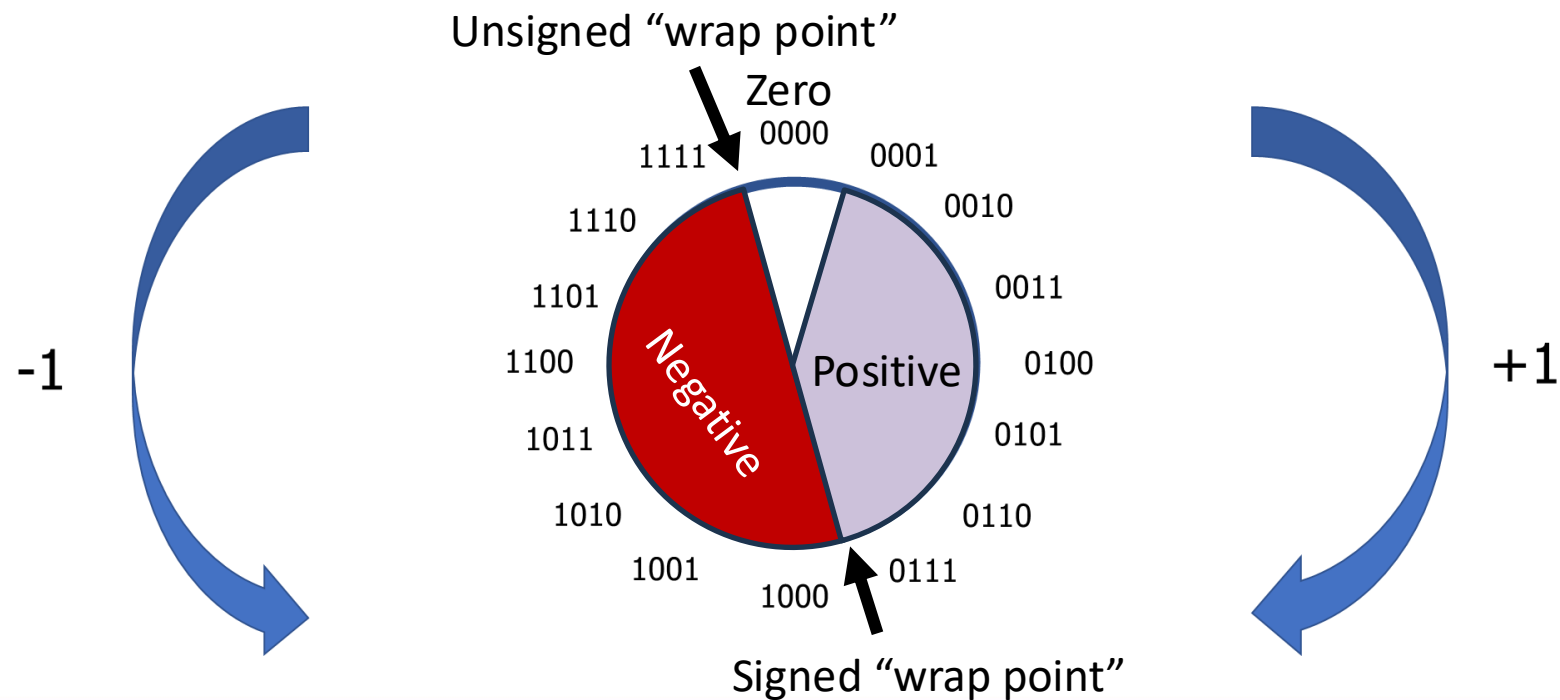
- Truncation makes signed addition work!
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{r} \overset{1}{1} \overset{1}{1} \overset{1}{1} \overset{1}{1} \\ 1101 \quad (-3) \\ + 0011 \quad (+3) \\ \hline 10000 \end{array}$$

- $-3_{10} + 3_{10} = 0_{10}$
 - Too large for 4 bits! Drop the carry bit
 - Result is what we expect as long as we truncate

Overflow of signed values

- Addition can lead to **overflow**
 - Unsigned: Result wraps around from big to small or vice versa
 - Signed: Both operands are positive and result is negative
 - Signed: Both operands are negative and result is positive



Signed addition and overflow

- Overflow can still happen in signed addition though
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{r} \overset{1}{0}\overset{1}{1}\overset{1}{0}1 \\ + \quad 0011 \\ \hline 1000 \end{array}$$

****For a fixed width w:**

- *Smallest value, most negative is 1 followed by all 0s bit pattern ($100\dots 0 = -2^{w-1}$)*
- *Largest value, most positive is 0 followed by all 1s bit pattern ($01\dots 1 = 2^{w-1} - 1$)*

- $5_{10} + 3_{10} = -8_{10}$ (+8 is too big to fit in 4 bits using a signed representation)
- Remember, this was also unsigned $5_{10} + 3_{10} = 8_{10}$

Signed addition and negative overflow

- Overflow also happens in the negative direction
- **Example: Adding two 4-bit signed numbers**

$$\begin{array}{r} \overset{1}{1} \overset{1}{1} \overset{1}{1} \\ 1011 \\ + 1011 \\ \hline 10110 \end{array}$$

****For a fixed width w:**

- *Smallest value, most negative is 1 followed by all 0s bit pattern ($100\dots0 = -2^{w-1}$)*
- *Largest value, most positive is 0 followed by all 1s bit pattern ($01\dots1 = 2^{w-1} - 1$)*

- $-5_{10} + -5_{10} = +6_{10}$ (-10 was too small to fit in 4 bits using a signed representation)

Overflow: hardware vs C standard

- Hardware implementations for unsigned and signed addition are the same
 - Both implement truncation of overflowing bits, leads to modular arithmetic
- Unsigned overflow in C is defined as modular arithmetic
 - Safe to use. Big number wraps around to a small number
- Signed overflow in C is **undefined behavior**
 - Compiler *probably* does modular result
 - But there are no promises about this, and it can make *assumptions*
 - So don't rely on it
 - Generally: overflow is bad and is to be avoided!

Negating a number

- In C:
 - $x = -y;$
- Operation
 - Determine the negative, signed version of the number (two's complement)
 - Hardware method: flip bits and add one
- Complement operator (~)
 - This is a bit-level operator
 - Flips all bits: zeros become a one and ones become a zero
 - $\sim 0b1011 \rightarrow 0b0100$

Negating via Complement & Increment

- Claim: The following is true for 2's complement

- $\sim x + 1 == -x$

$$\begin{array}{r}
 \mathbf{x} \quad \boxed{1} \boxed{0} \boxed{0} \boxed{1} \boxed{1} \boxed{1} \boxed{0} \boxed{1} \\
 + \quad \sim \mathbf{x} \quad \boxed{0} \boxed{1} \boxed{1} \boxed{0} \boxed{0} \boxed{0} \boxed{1} \boxed{0} \\
 \hline
 -1 \quad \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1}
 \end{array}$$

- Complement

- Observation: $\sim x + x == 1111...11_2 == -1$

- Increment

- $\sim x + 1 == \sim x + x - x + 1 == -1 - x + 1 == -x$

- Example, 4 bits: $6_{10} = 0110_2$

- Complement: $1001_2 \rightarrow$ Increment = $1010_2 = -8 + 2 = -6_{10}$

****Note: $x - x = 0$**

Subtraction in two's complement

- Subtraction becomes addition of the negative number
 - $5 - 3 = 5 + -3 = 2$
- Both unsigned and signed subtraction
 - Convert subtrahend to its two's complement negative form
 - i.e., negate it
 - Then do addition
 - Treat result as original encoding

$$\begin{array}{r} \overset{1}{0}101 \quad (+5) \\ + \overset{1}{1}101 \quad (-3) \\ \hline \textcolor{red}{1}0010 \end{array}$$

C rules vs hardware rules

- Exact same overflow rules apply
- Unsigned subtraction overflow can wrap below zero to make a large number
 - Modular arithmetic
- Signed subtraction overflow is **UNDEFINED BEHAVIOR**
 - And therefore should not be trusted

Practice

- Adding two 8-bit binary numbers:
 - Also determine the decimal version of the result
 - Omitting the encoding is intentional

$$\begin{array}{r} 00010101 \\ + 10110001 \\ \hline \end{array}$$

Practice

- Adding two 8-bit binary numbers:
 - Also determine the decimal version of the result
 - Omitting the encoding is intentional

	Unsigned encoding		Signed encoding
$\begin{array}{r} \overset{1}{0}\overset{1}{0}\overset{1}{1}\overset{1}{0}\overset{1}{0}\overset{1}{1} \\ + \underline{10110001} \\ \hline 11000110 \end{array}$	$\begin{array}{l} 16+4+1 = 21 \\ 128+32+16+1 = 177 \\ 128+64+4+2 = 198 \end{array}$	OR	$\begin{array}{l} 16+4+1 = 21 \\ -128+32+16+1 = -79 \\ -128+64+4+2 = -58 \end{array}$

Practice

- Adding two 8-bit binary numbers:
 - Also determine the decimal version of the result
 - Omitting the encoding is intentional

	Unsigned encoding		Signed encoding
$\begin{array}{r} 111 \\ 00010101 \\ + 10110001 \\ \hline 11000110 \end{array}$	$16+4+1 = 21$ $128+32+16+1 = 177$ $128+64+4+2 = 198$	OR	$16+4+1 = 21$ $-128+32+16+1 = -79$ $-128+64+4+2 = -58$

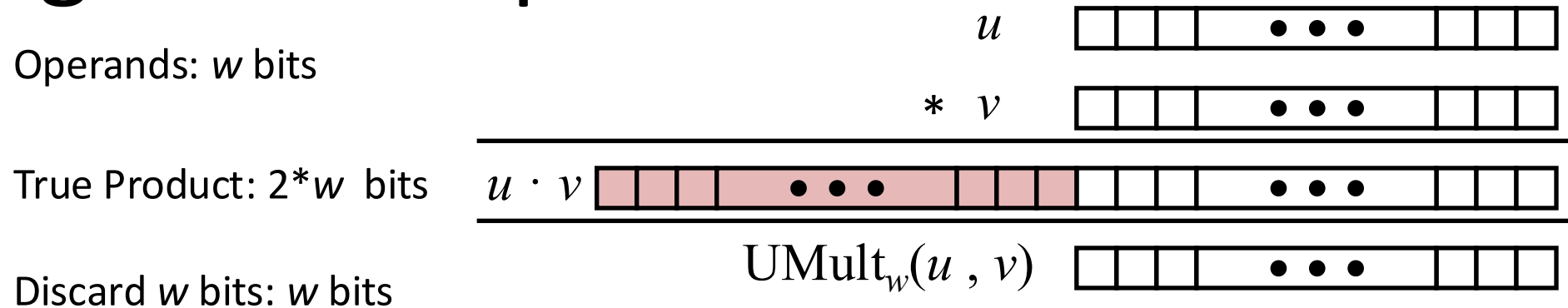
What about unsigned subtraction 21-79?

That would treat the result as unsigned, with the value 198. Modular arithmetic in action!!

Multiplication

- Goal: Compute the Product of two **w**-bit numbers x, y
 - Either signed or unsigned
- But, exact results can be bigger than **w** bits
 - Double the size ($2w$), in fact!
 - Example in base 10: $50_{10} * 20_{10} = 1000_{10}$
 - (2-digit inputs become a 4-digit output!)
- As with addition, result is truncated to fit in **w** bits
 - Because computers are finite, results can't grow indefinitely

Unsigned Multiplication



- **Standard Multiplication Function**
 - Equivalent to grade-school multiplication
 - But ignores most significant w bits of the result
 - As a person, we can do base 10 multiplication, convert to base 2, then truncate
- Implements modular arithmetic like addition does

$$\text{UMult}_w(u, v) = (u \cdot v) \bmod 2^w$$

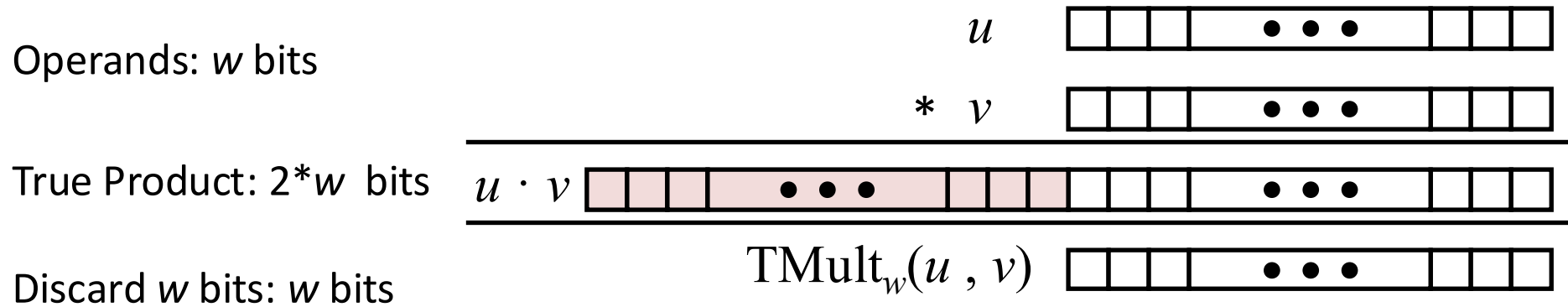
Unsigned multiplication

- Example: Multiplying two 4-bit numbers

$$\begin{array}{r} 0010 \\ \times 0101 \\ \hline 0010 \\ 00000 \\ 001000 \\ + 0000000 \\ \hline \cancel{000}1010 \end{array}$$

$$2_{10} * 5_{10} = 10_{10} \checkmark$$

Signed (2's Complement) Multiplication



- **Standard Multiplication Function** (same as unsigned)
 - Ignores most significant w bits
 - Lower bits still give the correct result
 - So we can use same machine instruction for both!
 - Again, that's one reason why 2's complement is so nice
- **In C, signed overflow is undefined**
 - ...but probably you'll see the two's complement behavior

Signed multiplication

- **Example: Multiplying two's complement 5-bit numbers**

$$\begin{array}{r} 11110 \quad -2 \\ \times 00011 \quad 3 \\ \hline 11110 \\ + 111100 \\ \hline \textcolor{red}{10}11010 \end{array}$$

What are these two 5-bit numbers?

What is the result of this addition?

$$-2_{10} * 3_{10} = -6_{10} \checkmark$$

What about divide?

- Annoying operation, not going to discuss in this class
 - Similar to long division process
 - Tedious and complicated to get right
- There are some computers that don't have hardware support for division at all!!
- Important thing to remember is that integers don't have fractional parts
 - In C: $1/2 == 0$
 - We'll need a different encoding for fractional numbers: floating point